

TestVDB: Detecting API Compliance Defects in Vector Database Systems via Contract-Truth Separation

Anonymous Authors
Paper #XXX

Abstract

Vector Database Management Systems (VDBMSs) have become critical infrastructure for Large Language Model (LLM) applications such as retrieval-augmented generation. A recent empirical study shows that *incorrect-behavior* defects account for 43% of VDBMS bugs—far exceeding crash/hang bugs (23%)—yet the community lacks practical test oracles for this majority class: state-of-the-art fuzzers such as VDBFuzz rely on crash oracles and cannot detect non-crash violations. We present **TestVDB**, the first LLM-driven system that detects *API compliance defects* in VDBMSs: bugs where the system silently accepts inputs or produces behaviors that violate its documented contract. TestVDB derives a contract oracle automatically from official API documentation using LLMs, and applies **Contract-Truth Separation** (CTS): it isolates the assertion layer (LLM-generated contracts and compliance judgments) from a truth layer that falsifies assertions through maintainer-authority evidence (source grounding, clean reproduction, by-design intent). The counter-evidence layer is necessary because we identify and characterize *contract hallucination propagation*: when an LLM both generates a contract and judges compliance, hallucinated constraints get self-confirmed in the generation-judgment chain. In our study, 25% of submitted issues were marked by maintainers as by-design because the LLM-derived contract was stricter than the true intended behavior. Across five VDBMSs, TestVDB produced 111 candidate issues; maintainers acknowledged 36 (28 fixed, 8 accepted), raising end-to-end precision from 12.9% (single-layer LLM judgment) to 69.2% (5.4×). We state the system’s boundary honestly: it specializes in boundary/validation compliance defects (75% of yield), explicitly excludes crash bugs (complementary to VDBFuzz), and does not solve the result-correctness oracle (ANN recall, ranking), which remains open.

1 Introduction

VDBMSs such as Milvus, Qdrant, Weaviate, Chroma, and MeiliSearch have become foundational infrastructure for data-intensive AI workloads, powering retrieval-augmented generation, recommendation, and multimodal search. As VDBMSs enter production at scale, their reliability directly affects downstream decisions. A large-scale empirical study of 1,671 bug-fix pull requests across 15 VDBMSs [?] finds that more than half of all defects manifest as functional failures; a complementary roadmap study [?] reports that *incorrect-behavior* bugs (43.0%) substantially outnumber crash/hang bugs (23.1%).

Despite this distribution, current VDBMS testing is skewed toward the minority. VDBFuzz [?], the first dedicated VDBMS fuzzer, uses crash as its oracle and detects only crash/hang bugs. It leaves the majority—non-crash violations of expected behavior—without

Table 1: Oracle candidates for API compliance defects (exclusion argument).

Oracle candidate	Why it fails for compliance defects
Crash (VDBFuzz)	Compliance defects do not crash
PoC reproduction	No observable failure signal
Differential testing	No reference implementation for VDBMS APIs
Equivalence transformation	No query form to transform equivalently
LLM	Only flexible semantic candidate—but generates and judges, both unreliable

a practical oracle. The roadmap [?] explicitly flags this as the central open challenge: “*incorrect behavior poses significant challenges for test oracles*,” and notes that approximate vector search makes traditional differential testing unsuitable. Its Future Work further asks for “an oracle for evaluating the correctness of vector search results” and for methods that let LLMs “stay updated with the latest syntax and functionalities.”

We target a tractable and previously unsystematized slice of this gap: *API compliance defects*—bugs where the VDBMS silently accepts an input or produces a behavior that violates its documented API contract (e.g., accepting `nprobe=0`, silently normalizing `shardsNum=-1`, or returning success for an operation the docs prescribe to reject). These bugs do not crash, but they corrupt query semantics, lower recall, and expand the attack surface. They admit an oracle where general result-correctness does not: the API contract itself.

Why LLM, and why that is not enough. For compliance defects there is no observable failure signal (no crash, no exception), and no reference implementation against which to differential-test. By exclusion (Table 1), the only candidate oracle capable of flexible semantic judgment over an undocumented boundary is an LLM. But using an LLM as the oracle introduces two unreliability roles: it *generates* the contract (from docs) and it *judges* compliance—and both can hallucinate.

Core reversal: the oracle itself may be wrong. The naive design—LLM extracts a contract from docs, then the same (or similar) LLM judges compliance against it—is brittle because of *contract hallucination propagation*. We observed a concrete instance: the contract formalizer “extracted” a complexity requirement from `constant.go` that is in fact a guard against a historical performance regression, not a behavioral constraint; the judge then confirmed violations against this hallucinated constraint. In our submissions, 12 of 48 maintainer-adjudicated issues (25%) were marked by-design: the LLM-derived contract was stricter than the maintainers’ true

intent (e.g., demanding rejection of idempotent duplicate creates, eventual-consistency staleness, or lenient defaults). When the same model family generates the contract and judges compliance, hallucinated constraints are self-confirmed.

Approach: Contract-Truth Separation. Our key insight is that when no mechanical oracle is available, the only available truth source is *maintainer authority*—source code, prior PRs, by-design intent, and issue history. It is scarce, so it must be proxied by an agent. We separate the assertion layer (LLM-generated contracts and judgments) from a truth layer, and use the latter to falsify the former. The truth layer is a *dev-reviewer* agent that performs counter-evidence against each candidate defect along three anchors: clean reproduction, source-grounded verification (the direct counter to contract hallucination), and threat-model cross-check. This mirrors how a human maintainer adjudicates an incoming bug report.

Results. We implemented TestVDB and ran it on five VDBMSs. It produced 111 candidate issues; maintainers acknowledged 36 (28 fixed, 8 accepted-open). The dev-reviewer layer alone removes 80.6% of false positives at the Stage-2 judgment layer, and end-to-end precision rises from 12.9% (single-layer LLM judgment, 4/31 on a Milvus ablation) to 69.2% (36/52, 5.4×). We report the system’s boundary honestly: 75% of its yield is boundary/validation compliance; crash bugs are excluded by design (complementary to VDBFuzz); and soft result-correctness (ANN recall, ranking) remains open.

Positioning relative to VDBFuzz. We respond to the generation-side Future Work of VDBFuzz [?] (LLM-generated diverse API interactions + staying current with evolving APIs). Our judgment side provides a contract oracle that extends detection beyond crash into the API-compliance subset of incorrect behavior. We do *not* claim to solve VDBFuzz’s oracle-for-correctness direction (result correctness of vector search), which remains open [?].

Contributions.

- (1) **First LLM-driven realization and large-scale empirical study of VDBMS API-compliance defect detection.** On the VDBMS-testing agenda defined by [?], we are the first to end-to-end realize detection of non-crash API-compliance defects and validate at scale (111 issues, 36 acknowledged).
- (2) **Contract-Truth Separation**, a design principle that isolates LLM-generated assertions from a maintainer-authority truth layer used to falsify them.
- (3) **A counter-evidence mechanism** (dev-reviewer) with three anchors, removing 80.6% of false positives and lifting precision 5.4×; we report its TP-recall limitation (20–60%) honestly.
- (4) **The contract hallucination propagation finding:** when an LLM both generates a contract and judges compliance, hallucinated constraints are self-confirmed—empirically observed in 12 by-design cases (25% of adjudicated submissions). The source-grounded counter-evidence layer is the mitigation.
- (5) **An exploratory pilot** of a threat-model prior, with honest design limitations.
- (6) **Empirical results and an open-source system:** 111 issues across 5 VDBMSs, 36 maintainer acknowledgments; system open-sourced.

Table 2: TestVDB’s scope projected onto the roadmap symptom taxonomy [?].

Roadmap symptom class	TestVDB coverage
Crash/Hang (23.1%)	<i>Out of scope</i> (L1 gate filters; 1 incidental)
Incorrect Behavior (43.0%)	In scope: boundary/validation compliance subset (75% of yield), plus diagnostic and state/logic subsets
Performance (9.3%)	Out of scope
Build (9.7%)	Out of scope

2 Background and Problem Formulation

2.1 VDBMS Architecture

A VDBMS stores, indexes, and queries dense vector embeddings. Typical layers [?] include a storage engine, a vector index (HNSW, IVF, etc.), a query processor (supporting search, filtered/predicated search, multi-vector search), and client SDKs exposing REST and gRPC APIs. Our defect targets live at the API surface and the query-processing logic behind it.

2.2 Bug Taxonomy and Our Scope

We adopt the symptom-level taxonomy of the VDBMS testing roadmap [?] as our primary frame: Crash/Hang (23.1%), Incorrect Behavior (43.0%), Performance (9.3%), Build (9.7%). We corroborate it with the larger empirical study [?] (5 symptom categories, 31 fault patterns) used as a fault-pattern vocabulary. Table 2 states our scope by projecting our compliance-dimension axis onto the symptom axis.

2.3 API Compliance Defects

We define an *API compliance defect* as a behavior in which the VDBMS, for an input governed by a documented or implementable contract, either (i) accepts an input the contract prescribes to reject (*illegal success*), (ii) rejects or mishandles an input the contract prescribes to accept, or (iii) returns a response whose status, payload, or side effects violate the contract (*poor diagnostics / state violation*). The contract is not assumed perfect; Section 4 addresses contract unreliability directly.

2.4 The Oracle Problem

For compliance defects there is no crash, no exception, and—unlike SQL—no widely-agreed reference semantics for VDBMS APIs across vendors. Differential testing is unsuitable because APIs diverge and results are approximate [?]. Metamorphic relations help for result correctness but not for API-acceptance decisions. The contract itself is the only candidate oracle—but the contract must be obtained and trusted, which is the core problem we address.

3 Approach

3.1 Overview

TestVDB is a five-stage pipeline (Figure 1): (1) contract extraction from API documentation via an LLM *knowledge-extractor*; (2) attack generation by specialized agents driven by a threat-model

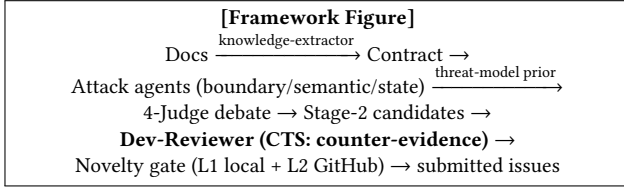


Figure 1: TestVDB pipeline. The dev-reviewer is the core contribution; the threat-model prior spans generation, judgment, and dedup.

prior; (3) a four-judge debate producing Stage-2 candidates; (4) a dev-reviewer that applies Contract-Truth Separation as counter-evidence; (5) a two-layer novelty gate. A threat-model artifact is reused across stages as an experience prior.

3.2 Contract Extraction and the Threat-Model Prior

A knowledge-extractor agent crawls official API documentation and emits structured contracts (parameter domains, enum values, documented limits, status-code semantics). These contracts seed both the attack agents and the judges. A threat-model artifact encodes recurring by-design intents (idempotency, eventual consistency, lenient defaults) and known blind spots; it is injected as a prior at generation (to steer attacks toward non-trivial boundaries), at judgment (as a counter-evidence dictionary for the dev-reviewer), and at dedup. We treat the threat model as an *exploratory pilot* (Section 5.4) rather than a core claim, because our ablation could not cleanly isolate its effect.

3.3 Attack Generation and the Four-Judge Debate

Three attack agents target the compliance dimensions: a *boundary* agent (illegal-success at parameter/enum/limit edges—the dominant yield, cf. Table 3), a *semantic* agent (diagnostic-quality violations), and a *state* agent (state/logic inconsistencies). Generated candidates enter a four-judge debate (evidence/severity/novelty/doc axes). The Stage-2 output of this debate is the input to the dev-reviewer.

3.4 Contract-Truth Separation and the Dev-Reviewer

The single-layer LLM judgment above is unreliable because the same model family generates the contract and judges compliance. CTS breaks this self-confirmation by introducing an independent truth layer. The dev-reviewer agent acts as a proxy for maintainer authority and falsifies each Stage-2 candidate along three anchors:

- **Clean reproduction.** Construct a minimal reproducer against a live VDBMS of the pinned target version; reject candidates whose “defect” is a tooling artifact (we withdrew early candidates that misread a missing `rowCount` field as “returns -1”).
- **Source-grounded verification.** Locate the implementation behind the API and check whether the observed behavior is intended—the direct counter to contract hallucination (Section 4).

- **Threat-model cross-check.** Match against known by-design intents before flagging a violation.

This mirrors maintainer adjudication: most false positives we observe stem from contracts stricter than true intent, and source grounding is what exposes them.

3.5 Novelty Gate

A two-layer gate prevents re-reporting: L1 checks against local history and the threat-model blind-spot set; L2 queries the target repository’s issues/PRs. Candidates surviving both are submitted.

4 Contract Hallucination Propagation

We highlight a phenomenon that, to our knowledge, has not been characterized in LLM-driven testing. When an LLM generates the contract *and* judges compliance, hallucinated constraints are not caught by the judge—they were produced by the same generation-judgment family and thus inherit mutual confirmation. We observed two forms:

(1) **Fabricated provenance.** The formalizer produced a “complexity requirement” attributed to `constant.go`; source grounding showed the constant guards a past performance regression, not a behavioral constraint. The Stage-2 judge nonetheless confirmed violations against it.

(2) **Over-strict intent.** 12 of 48 maintainer-adjudicated submissions (25%) were marked by-design: the contract demanded rejection of behaviors maintainers intended to allow—idempotent duplicate creates returning success, eventual-consistency staleness, lenient naming defaults, and silent enum/empty substitutions.

Formally, let C_{LLM} be the LLM-derived contract and C_{true} the maintainer intent. Single-layer judgment treats $C_{LLM} = C_{true}$; when $C_{LLM} \supset C_{true}$ (stricter), genuine by-design behaviors are flagged as violations, and the judge—sharing the generator’s bias—does not dissent. The dev-reviewer’s source-grounded anchor is the mitigation: it reintroduces an external truth signal (C_{true} proxied by source + history). We position the empirical measurement of hallucination frequency as future work, but the 12 by-design cases are direct observations of the phenomenon.

5 Evaluation

We address four research questions.

5.1 RQ1: Detection Capability

We ran TestVDB against five VDBMSs (Milvus, Qdrant, Weaviate, MeiliSearch, Chroma), pinning exact target versions. Table 3 reports the yield.

Defect-type distribution (title-classified). Of the 36 acknowledged true positives, 27 (75%) are boundary/validation, 3 (8%) diagnostic-quality, 2 (6%) state/logic, 1 (3%) crash, and 3 (8%) result-correctness. The result-correctness cases are notable: they include COSINE distance returning > 1.0 for identical vectors (a hard mathematical-invariant violation, reproduced on both Milvus and Qdrant), incomplete index results (2 of 25 matching points returned), and a payload filter returning points with a missing field. These are detectable via contract because they violate expressible invariants; soft result-correctness (ANN recall/ranking error) remains open.

Table 3: Issues submitted and maintainer outcomes (5 VDBMSs).

VDBMS	Submitted	Fixed	Accepted	By-design	Rejected
Milvus	51	14	8	12	0
Qdrant	26	11	0	0	3
Weaviate	30	3	0	0	1
MeiliSearch	3	0	0	0	0
Chroma	1	0	0	0	0
Total	111	28	8	12	4

Complementarity with VDBFuzz. Our yield is almost entirely non-crash (35/36); VDBFuzz targets crash bugs. The near-zero overlap is by design (our L1 gate filters crash-prone inputs). We frame the two systems as complementary, not competitive.

5.2 RQ2: Case Studies

Contract hallucination (Milvus). The formalizer fabricated a complexity requirement; the dev-reviewer’s source-grounded anchor traced it to `constant.go` and reclassified the candidate, preventing a spurious submission. **Dual-library cosine invariant.** The COSINE distance > 1.0 for identical vectors was independently reproduced on Milvus and Qdrant, confirming a cross-vendor correctness class amenable to contract oracles.

5.3 RQ3: Precision Ablation

On a Milvus v2.6.19 ablation, 1,834 raw candidates collapsed to 31 Stage-2 outputs, of which 4 were true positives—a single-layer LLM-judgment precision of 12.9% (4/31). Adding the dev-reviewer removes 25/31 false positives (80.6%). End-to-end across all five VDBMSs, precision is

$$\text{precision} = \frac{\text{acknowledged}}{\text{acknowledged} + \text{by-design} + \text{rejected}} = \frac{36}{36 + 12 + 4} = 69.2\%$$

a 5.4× improvement over the 12.9% single-layer baseline. We deliberately include *rejected* in the denominator (treating wontfix/not-planned as maintainer-judged false positives) for a conservative estimate.

5.4 RQ4: Threat-Model Pilot (Exploratory)

We ran a double-blind ablation on Milvus v2.6.17 (control with threat-model cognition vs. experiment without; $n = 5$ TP, 7 FP). TP recall moved 20%/60% (control/experiment); FP-correct 86%/57%. We *do not* claim a TM effect: blindspot indicators were never populated (so the TP-protection mechanism went untested), the dev-reviewer was unstable on the same candidate across runs (2/2 CONFIRMED/FP), and $n = 5$ is below significance. The reliable finding is orthogonal and humbling: the dev-reviewer’s TP recall is only 20–60%, i.e., even with counter-evidence, an LLM proxy struggles to recognize genuine bugs—a core limitation of CTS.

5.5 Threats to Validity

Internal: maintainer acknowledgment is a weak ground truth; rejected/by-design outcomes involve human judgment. *External:* cross-layer

ablation (RQ3) is Milvus-only; cross-library dev-reviewer data is incomplete (Weaviate has 22 open undiagnosed). *Construct:* title-based defect-type classification is approximate and pending per-defect label confirmation. *LLM variance:* dev-reviewer judgments vary across runs; we report ranges, not point estimates.

6 Related Work

VDBMS testing. VDBFuzz [?] is the first dedicated VDBMS fuzzer, crash-oracle-based and complementary to this work. The roadmap [?] and the empirical bug study [?] define the agenda and taxonomy we build upon. MeTMAP [?] applies metamorphic testing to false vector matching in LLM-augmented generation. BUZZBEE [?] fuzzes general DBMSs (relational and non-relational). **Database correctness oracles.** NoREC [?] tests SQL via negation-based oracles; DDLCheck [?] tests DDL correctness. These assume reference SQL semantics absent in VDBMS APIs. **LLM-based testing and multi-agent debate.** LLMs are increasingly used for test generation and oracle enhancement [?]; we use multi-agent debate for generation/judgment but contribute the counter-evidence (CTS) layer that addresses the contract-hallucination failure mode of self-confirming LLM oracles.

7 Conclusion and Future Work

We presented TestVDB, the first LLM-driven system for detecting API compliance defects in VDBMSs, built on Contract-Truth Separation. The dev-reviewer counter-evidence mechanism lifts precision 5.4× by falsifying LLM-generated assertions against maintainer authority, and its necessity is motivated by our characterization of contract hallucination propagation. We honestly bound the system: it specializes in boundary/validation compliance, is complementary to crash-focused fuzzing, and does not solve result-correctness oracles. Future work includes stronger dev-reviewer backbones (we plan to re-run triage of the 48 adjudicated issues with a stronger model to measure TP-recall vs. FP-suppression gains), cross-library ablation, and a quantitative study of contract hallucination frequency.

References